ToddySoft Connect ADS - Apache StreamPipes

An Apache StreamPipes **adapter** for Beckhoff TwinCAT controllers, built on the ToddySoft Connect ADS driver. It reads tags from Beckhoff TwinCAT 2 and TwinCAT 3 controllers over ADS/AMS, plain or as Secure ADS (TLS with a client certificate or a pre-shared key). Every polling interval produces one event in StreamPipes with the selected tags' values.

The adapter comes as its own **extension service**: a container that runs next to your StreamPipes installation, registers with it, and appears in StreamPipes as the adapter *ToddySoft Connect ADS*.

Requirements

- **Apache StreamPipes 0.99**, installed with Docker (the standard installer). The service joins the installation's Docker network.
- **Linux on the machine running StreamPipes**, amd64 or arm64, with a route to the devices. Docker Desktop (macOS, Windows) runs containers in a virtual machine: adapters work with *Enter address*, but no search reaches the plant network.
- A **ToddySoft licence for ADS** for production use. Without one the service runs on its bundled trial licence until that expires.

Installation

Loading the image

The service is delivered as an image archive, one per architecture, in the share you received your licence and downloads from: `streampipes-ads-<version>-<architecture>.tar.gz`, for `amd64` or `arm64`. Use the archive of the version you were sent and the architecture of the machine (`uname -m`: `x86_64` is amd64, `aarch64` is arm64).

```
docker load < streampipes-ads-1.3.0-amd64.tar.gz
```

> **The images are not on Docker Hub or any public registry**, and the same goes for the discovery agent: every image reaches you through this one channel. We plan to move to a private registry with a pull account per customer, so that `docker pull` and updates work without archives; until then, load each new version the same way.

Running the service

Start the service in the Docker network of your StreamPipes installation (`spnet` for the standard installer; `docker network ls` shows it):

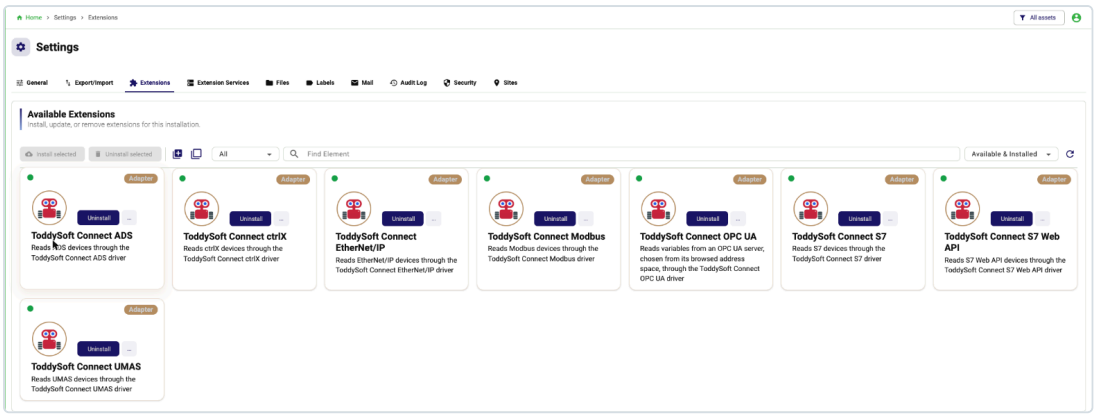
```
docker run -d --name toddysoft-ads --restart unless-stopped \
  --network spnet \
  -e SP_HOST=toddysoft-ads -e SP_PORT=8005 \
  -e SP_CLIENT_SECRET=<the SP_CLIENT_SECRET of your installation> \
  -e SP_PRIORITIZED_PROTOCOL=kafka \
  toddysoft/streampipes-ads:1.3.0
```

- **`SP_HOST`** must be the container's own name: StreamPipes calls the service back by it.
- **`SP_CLIENT_SECRET`** must be the value the StreamPipes core runs with (the installer's `.env`). With another value the service cannot register.
- **`SP_PRIORITIZED_PROTOCOL`** is your installation's broker: `kafka` for the standard installer.
- Keep the service on the installation's network only. It reaches the core and the broker there by name; host networking would lose them. The devices are reached through the machine's routing, and the search through the discovery agent (see Reaching the devices).
- Files the adapter needs - a keystore, a trust store - are mounted with `-v`, for example `-v /opt/toddysoft/certs:/certs:ro`, and given in the form as `/certs/<file>`.

`docker logs toddysoft-ads` should show the service registering with StreamPipes within a minute.

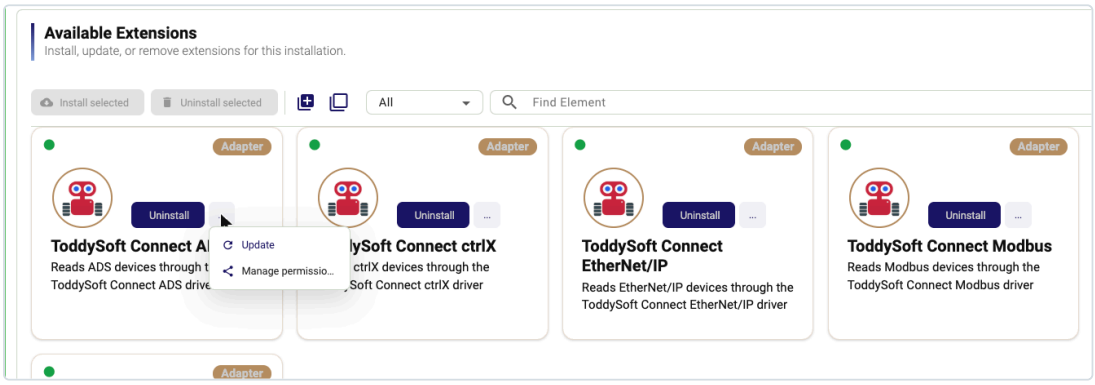
Installing the adapter in StreamPipes

Once the service has registered, install its adapter once: **Settings → Extensions**, find *ToddySoft Connect ADS* and press **Install**. It then appears under **Connect → New Adapter**.



Updating

1. Load the new archive and start the new version in place of the old container (`docker rm -f toddysoft-ads`, then the `docker run` above with the new version).
2. **Update the installed adapter**: Settings → Extensions → the adapter's **"…"** → **Update**. StreamPipes keeps the adapter description it stored at installation, so without this step the form of the previous version is shown, and new settings are missing.

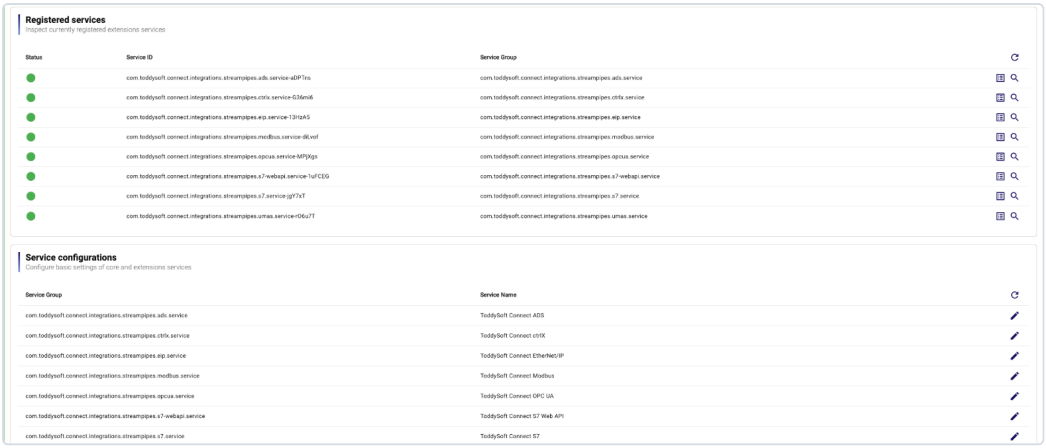


Adapters saved with the old version keep running after an update.

Licence

The driver reads its licence from the service's configuration in StreamPipes:

1. Open **Settings → Extension Services**. *Registered services* lists the running ToddySoft services; **Service configurations** below holds one entry per service.



2. Press the pencil next to *ToddySoft Connect ADS*. The entry has one field, `licence-ads`.



3. Paste the licence file **Base64-encoded**: `base64 -i ads.lic` (macOS) or `base64 -w0 ads.lic` (Linux) prints it.

4. **Click into the field once more after pasting**, then press **Update**. StreamPipes takes the field's value only on that click; saving straight after pasting stores the previous value, usually empty.

The licence takes effect on the next browse or adapter start; no restart is needed. Without a licence, or with one the driver rejects, the bundled trial applies until it expires. The value is masked in the form but stored in StreamPipes' database as entered, so limit who can administer StreamPipes.

Reaching the devices

A service in a container sees only its container networks. Connecting, browsing and reading go out through the machine's routing and work as long as the machine has a route to the device. **The search** does not: broadcasts, multicast and PROFINET-DCP never leave the container's network, so on its own the search finds nothing on the plant network. There are two ways to give it the plant network.

The ToddySoft discovery agent (recommended)

The agent is a small container that runs once per machine with the host's network and searches the plant networks for every ToddySoft service that asks. It only searches: the adapter still connects from its own container. It comes as an archive next to this service's, `discovery-agent-<version>-<architecture>.tar.gz`; use the agent of the same version.

```
docker load < discovery-agent-1.3.0-amd64.tar.gz
docker run -d --name toddysoft-discovery-agent --restart unless-
        stopped \
  --network host --cap-add NET_RAW \
  toddysoft/discovery-agent:1.3.0
```

- **Nothing to set on the service.** At every search it looks for the agent at its container network's gateway, which on Docker is the machine itself. An agent started later is found on the next Reload. The search's status line says *via discovery agent* when the agent answered, *no discovery agent found · container networks only* when there is none, or why the agent refused.
- **Licence.** The service asks only for ADS devices and sends the licence its driver runs on over an encrypted connection; the agent checks it, uses it for that one search and keeps nothing.
- **Security.** The agent listens only on the machine's Docker-network addresses (TCP 47900) and accepts connections only from the container networks; allow that port from them if the machine has a firewall. For more, set a shared secret: `-e TODDYSOFT_AGENT_SECRET=<secret>` on the agent and `-e TODDYSOFT_DISCOVERY_AGENT_SECRET=<secret>` on the service.
- **Discovering is not reaching.** A device the agent finds on a network the machine cannot route to is listed and then fails to connect.

A second network on the plant interface

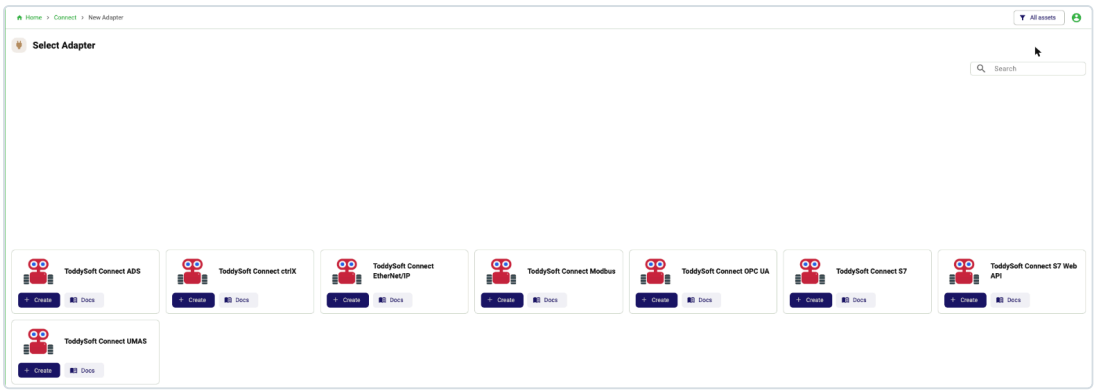
Alternatively, connect the service to a macvlan network on the plant interface, in the devices' own subnet, as a **second** network (it keeps `spnet`):

```
docker network create -d macvlan --subnet 192.168.23.0/24 --gateway
        192.168.23.1 -o parent=eth1 plant
docker network connect --ip 192.168.23.250 plant toddysoft-ads
```

Each service needs this, once per plant subnet. Set `-e TODDYSOFT_DISCOVERY_AGENT=off` on a service whose only network is a macvlan, so it does not look for an agent at the plant router.

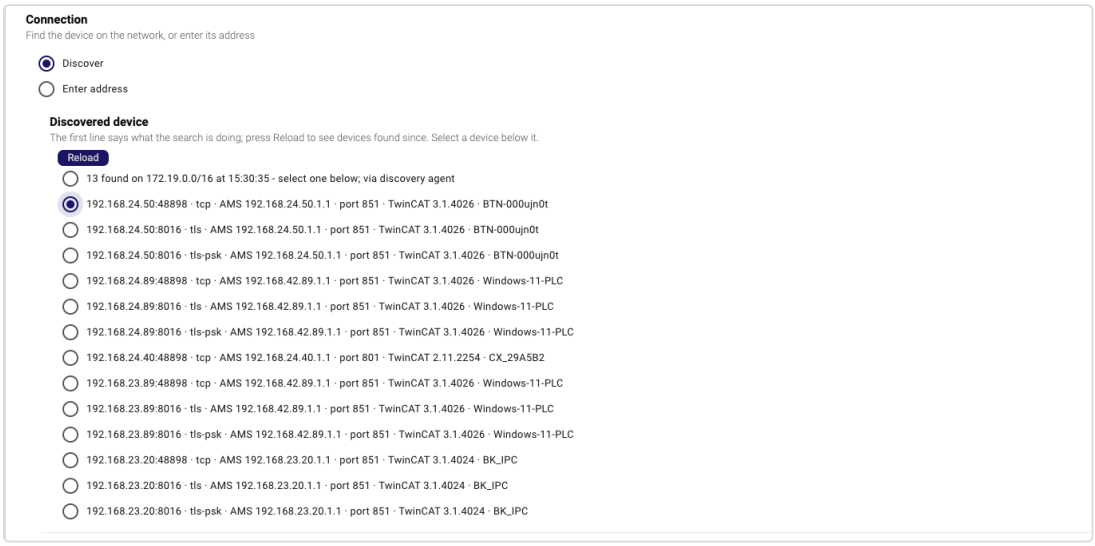
Creating an adapter

Open **Connect** → **New Adapter** and press **Create** on *ToddySoft Connect ADS*.



Finding the device

The form opens on **Connection → Discover**, and the search starts on its own.



The search is a broadcast: on the container's own networks, or with the discovery agent on the machine's plant networks. Each controller is listed **once per way it can be reached** - plain AMS, TLS, TLS-PSK - with its address, AMS Net ID and name. Choose the entry for the transport you want: **Transport security** further down must match it, because it supplies that transport's credential.

The first line of **Discovered device** says what the search is doing and which source answered (*via discovery agent*, or the container's own networks). It runs in the background for up to 25 seconds, so press **Reload** next to the field to see the devices found since. Select a device below that line: its address and connection settings are taken as found. A finished search is reused for a minute; Reload after that searches again. To type the address instead, choose **Enter address**.

The source AMS Net ID and the route

A TwinCAT controller answers only a client it has an **AMS route** for, and it recognises the client by its **source AMS Net ID**. In a container that is not the container's own address: the controller sees the plant machine.

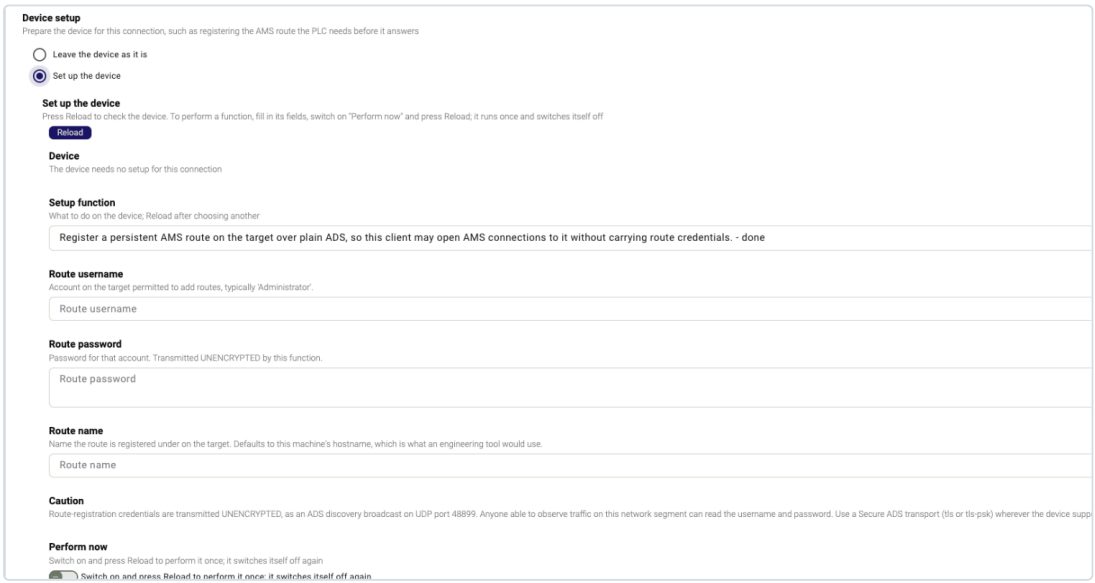


- **For a discovered device the field is filled in when you choose the device**: the address of this machine the device's answer arrived on, followed by `.1.1` - for example `192.168.24.251.1.1`.
- A value you typed yourself is kept when you choose another device; one filled in for another device is replaced.
- Change it when the route on the controller was created for another Net ID.
- Left empty, the driver uses `<the service's own address>.1.1`, which in a container is the container's address - a Net ID the controller cannot reach, and for which no route can work.

TwinCAT keeps one connection per source AMS Net ID. A second client on the same machine with the same Net ID - an Ignition gateway, another StreamPipes service - takes the connection away from the first one. Give each client its own Net ID and route.

Setting up the device: registering the route

A controller found by discovery often has no route for this machine yet. **Device setup → Set up the device** checks the connection exactly as entered above - address, AMS Net IDs, transport and credential - and says whether the device needs setup and why.



1. Choose **Set up the device** and press **Reload** next to the group. The check takes a few seconds; press Reload again while it says it is still checking.
2. **Setup function** lists what can be done for this connection - a plain AMS route, a TLS route, or the TLS-PSK route - each marked *needed* or *done*. Press Reload after choosing another one to see its fields.
3. Fill in the function's fields: for a route, the controller's administrator account. **Registering a plain route sends that password unencrypted**; a Caution line above *Perform now* says so.
4. Switch on **Perform now** and press **Reload**. The function runs **once**. The form comes back with *Perform now* switched off, the password field empty and a **Result** line: done, or the controller's reason for refusing. Nothing is retried, and nothing is sent to the controller without *Perform now*.

- **TwinCAT 2 needs up to about two minutes for a new route.** It restarts its AMS router and refuses connections meanwhile, so the check keeps saying the device needs setup. Do not perform again: press Reload in a minute or two. TwinCAT 3 applies a route at once.
- **A TLS-PSK route cannot be registered over the network**; the form shows the steps in TwinCAT's own router instead.
- When this service is already connected to the controller (the tag tree of this form, or a running adapter), the check reports no setup needed and does not connect again: a second connection with the same Net ID would be refused or would cut off the first.
- **If you typed the administrator password and did not perform the function, clear it before saving**: every field of the form is stored with the adapter. Choose *Leave the device as it is* when the route already exists; that is the default.

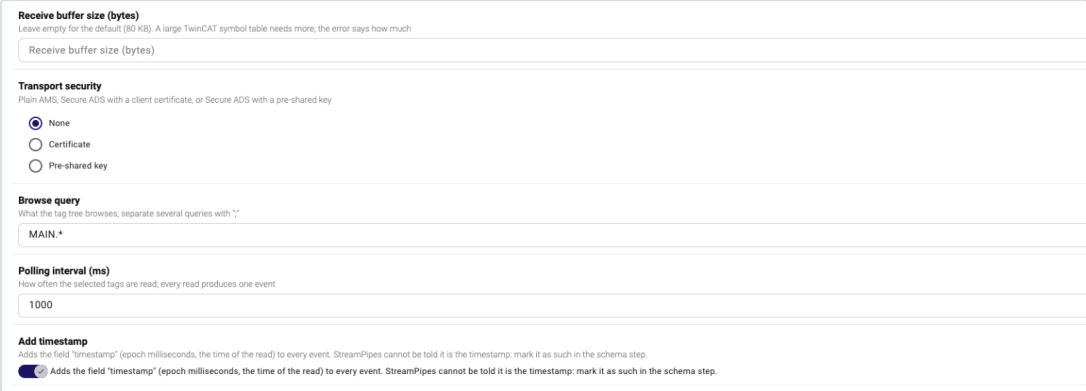
Secure ADS

Transport security chooses the transport:

- **None**: plain AMS on TCP 48898.
- **Certificate**: Secure ADS with this client's certificate from a PKCS12 keystore. **Keystore path** is a path *inside the service's container*: mount the file with `-v` when starting it (see Running the service). With **Verify the PLC's certificate** on, the controller's certificate is checked against the **trust store** (a shared CA); off, for a self-signed controller certificate.
- **Pre-shared key**: Secure ADS with a PSK identity and key, as configured on the controller.
- **Machine name** is announced to the controller in the Secure ADS session request. With a self-signed certificate it must equal the certificate's common name, or the controller answers `CnMismatch`.

The keystore password and the pre-shared key are handed to the driver as credentials, never written into the connection string.

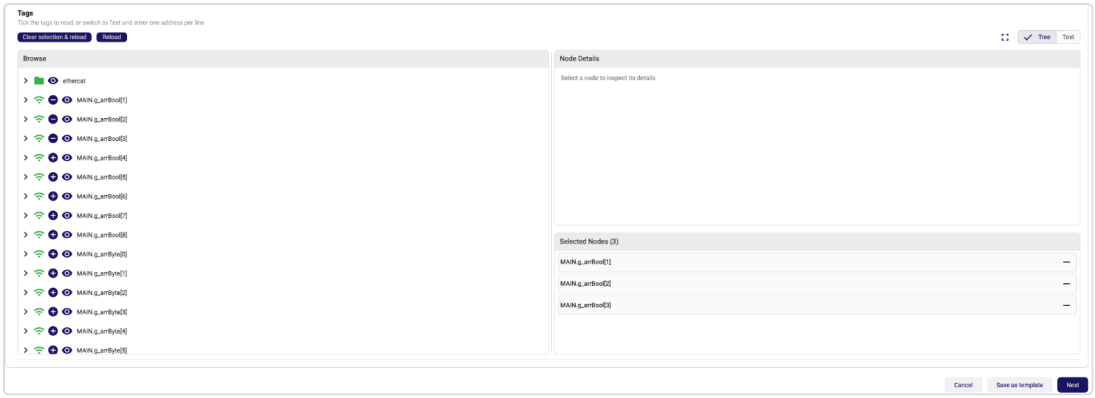
Settings



Setting	Meaning
Connection	*Discover* a device, or *Enter address* to type the fields below yourself
Device setup	*Leave the device as it is* (default) or *Set up the device*, see above
Host, Port	Address of the controller (Enter address only)
Target AMS Net ID	AMS Net ID of the controller, `x.x.x.x.x.x` (Enter address only)
Source AMS Net ID	This machine's Net ID as the route names it; see above
Source AMS port	Leave empty for the port of the connection itself
Target AMS port	The PLC runtime; empty for what discovery found (801 TwinCAT 2, 851 TwinCAT 3) or 851
Receive buffer size (bytes)	Empty for 80 KB; a large symbol table needs more, and the browse error names the size
Transport security	None, Certificate or Pre-shared key; see above
Browse query	What the tag tree shows, default `MAIN.*`; separate several queries with `;`
Polling interval (ms)	How often the selected tags are read; every read produces one event
Add timestamp	Adds the field `timestamp` (the time of the read) to every event; see below
Tags	What is read; see below

The **Documentation** link at the top of the form opens a short version of this page in StreamPipes.

Choosing the tags

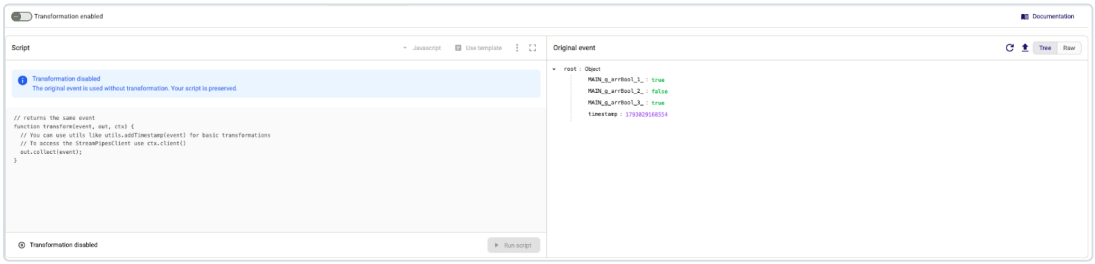


The tag tree shows what the **Browse query** returns. Press **+** next to a tag to read it, or on a folder to read everything below it; the eye shows a tag's type on the right, and **Selected Nodes** lists what will be read. **Reload** asks the device again; **Clear selection & reload** also empties the selection. Each tag becomes one field of the event, named after its address.

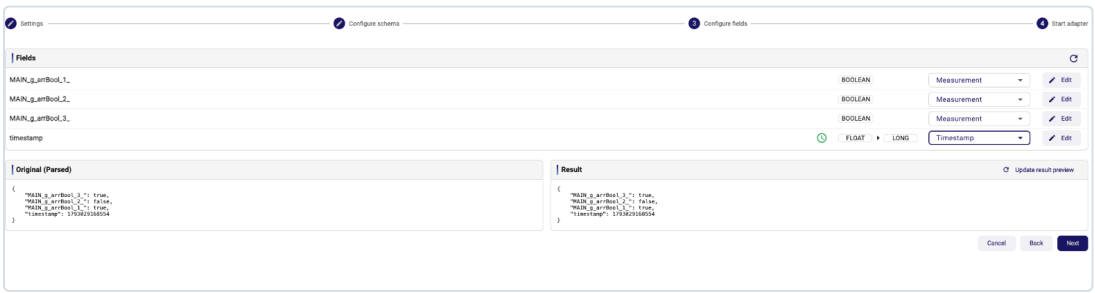
The tree's **Text** mode edits the selection directly, one address per line - useful to paste a list of addresses. StreamPipes puts its own comment lines into that editor (`# Provide OPC UA Node IDs …`); lines starting with `#` are ignored.

Events and the timestamp

Press **Next**. StreamPipes reads the tags once and shows the sample event; every poll then produces one event like it, with one field per tag that could be read and **`timestamp`**, the time of the read in milliseconds since the epoch.



StreamPipes cannot be told by the adapter which field is the timestamp. In **Configure fields**, set `timestamp`'s role to **Timestamp** and its type to **Long**:



- StreamPipes 0.99 types **every number as a float** in the schema it infers. The events themselves carry the exact values; change a field's type here where it matters.
- A tag that is itself named `timestamp` arrives as `timestamp_tag`.
- **Add timestamp** off leaves the field out, for a pipeline that sets its own.

Then name the adapter and start it under **Start adapter**.

Troubleshooting

- **The form looks like an older version, or a setting described here is missing**: update the installed adapter (Settings → Extensions → "…" → Update).
- **The tag tree shows an error**: the message names the cause - an unreachable address, a refused login, a missing licence. Where it says only *An unknown error has occurred*, the service's log has the details: `docker logs toddysoft-ads`.
- **The search finds nothing**: read its status line. *Container networks only* means neither the discovery agent nor a plant network is there (see Reaching the devices).
- **The licence is not taken**: click into the field after pasting, then Update. A licence the driver rejects is logged as `blob rejected for driver`, and the trial applies.
- **The service does not appear under Extensions**: check `docker logs toddysoft-ads`; usually `SP_CLIENT_SECRET` or the network is wrong.
- **"the device did not answer"** or **"the device closed the connection"**, each followed by advice to create the route: the controller has no route for the source AMS Net ID. TwinCAT 3 stays silent, TwinCAT 2 hangs up. Check the Net ID under *Source* and register the route with *Device setup*.
- **"Connection closed by remote" within milliseconds of connecting**: another client on this machine uses the same source AMS Net ID (see above).
- **TwinCAT 2: the route was registered and connecting still fails** for up to two minutes while the controller restarts its router.

Version history

Unreleased — 1.3.0

Built on ToddySoft Connect 1.3.0.

- **New:** the first release of the ADS adapter for Apache StreamPipes 0.99 - device search, the tag tree, polling into StreamPipes events, the licence in the service configuration, and search from containers through the ToddySoft discovery agent.